```
    (deleted, shouldPrune) ← delete(node.children[c],
tail)                    #7
    if deleted and shouldPrune
then                                                  #8
      node.children[c] ← null
                                                #9
      if node.keyNode == true or node.children.size > 0
then                    #10
        shouldPrune ←
false                                                    #11
    return (deleted, shouldPrune)
  else
    return (false,
false)                                                     #12
```

#1 Method `remove` takes a node and the string to delete from the sub-tree rooted at `node`. It returns a couple of Booleans: the first one tells the caller if the key has been successfully deleted, the second one is true if the last link followed became a dangling empty branch and should be pruned.

#2 Check if the string searched is empty: if it is, we traversed the whole path till the final node for the string to delete.

#3 We make sure that current node is marked as an intermediate node.

#4 Search is over, so return to the caller, reporting if the operation was successful (we had saved the value for `deleted` a couple of lines above) and if this node is to be pruned (in case it's a leaf).

#5 Since `s` is not the empty string, it can be broken into a *head* character, `c` (key first character of `s`) and a *tail*, the rest of the string. (Remember that, performance-wise, you